

The DCI: an acquired company inside the VPN

Topology, the path a branch takes, the routes exchanged, the twice-NAT, the DNS fix-up — and how the test suite proves all of it

Catalyst 8000v IPsec VTI lab · github.com/dcantor/cat8000v-ipsec . Every address, configuration line and command output in this document was taken from the running lab; the test results are from run `results/2026-09-23_02-08-05` (74/74 passing).

Contents

1. What the DCI is, and the problem it solves

2. Topology and the objects behind it

3. The path a branch takes to the acquisition

4. The routes exchanged (and the ones deliberately not)
5. How the NAT is set up

6. How DNS is fixed up

7. How the test is run and validated

8. Where every line of this comes from

1. What the DCI is, and the problem it solves

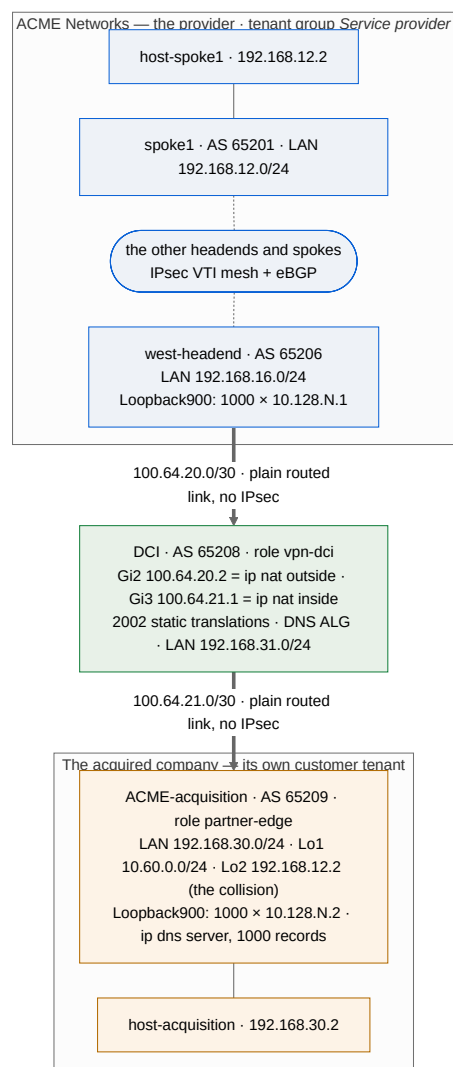
ACME Networks sells the VPN service; the branches are its customers. **ACME has acquired a company**, and that company arrives with its own network — including addresses ACME already uses. The lab models the whole situation: a **data-centre interconnect** (DCI) hangs off one headend, the acquired company's edge router (ACME-acquisition) hangs off the DCI, and the two sides collide on **1001 prefixes** at once:

Collision	On the ACME side	On the acquired company's side
A branch LAN, reused verbatim	192.168.12.0/24 — branch 1's LAN; host-spoke1 is 192.168.12.2	192.168.12.0/24 — a server on Loopback2, also 192.168.12.2
A thousand /24s (the scale set)	10.128.0.0/24 ... 10.131.231.0/24, ACME holds .1 in each	the same thousand prefixes, the acquired company holds .2 in each

Nothing is renumbered. The DCI hides the collision with a **twice-NAT**, keeps each side's copy on its own side with a route filter, and **doctors DNS answers** in flight so that a name resolved across the DCI returns an address the asker can actually reach. The whole arrangement is modelled in Nautobot, rendered by the same Network-as-Code pipeline and Golden Config template as every other router, and proven by twelve Robot tests.

Why it is interesting. The scale lands on the NAT, not on the routing table: the DCI carries **2002 static translations** while BGP carries **three aggregates**. A thousand interfaces per side would be absurd, so each side holds its thousand addresses as **secondary addresses on one loopback**.

2. Topology and the objects behind it



Device	Nautobot role	Tenant	AS	Interfaces that matter
west-headend	vpn-hub	ACME Networks	65206	Gi3 100.64.20.1 → the DCI · Gi9 LAN · Loopback900 10.128.N.1 ×1000
DCI	vpn-dci	ACME Networks	65208	Gi2 100.64.20.2 nat outside · Gi3 100.64.21.1 nat inside · Gi9 LAN 192.168.31.0/24
ACME-acquisition	partner-edge	(its own customer tenant)	65209	Gi2 100.64.21.2 · Gi5 LAN 192.168.30.1 · Lo1 10.60.0.1/24 · Lo2 192.168.12.2 · Loopback900 10.128.N.2 ×1000

Both links are cables between router ports in Nautobot — no firewall between them — and neither router carries a tunnel, a crypto profile or a keyring: **the encryption stops at the headends**. The acquired company's overlapping prefix lives in its own Nautobot **namespace** (`acquisition`), because it is not ACME's address space.

3. The path a branch takes to the acquisition

A branch has no special knowledge of the DCI. It learns the acquisition's prefixes over its IPsec tunnels like any other destination, and the path runs branch → tunnel → headend(s) → west-headend → DCI → acquisition. Taken live from `host-spoke1`:

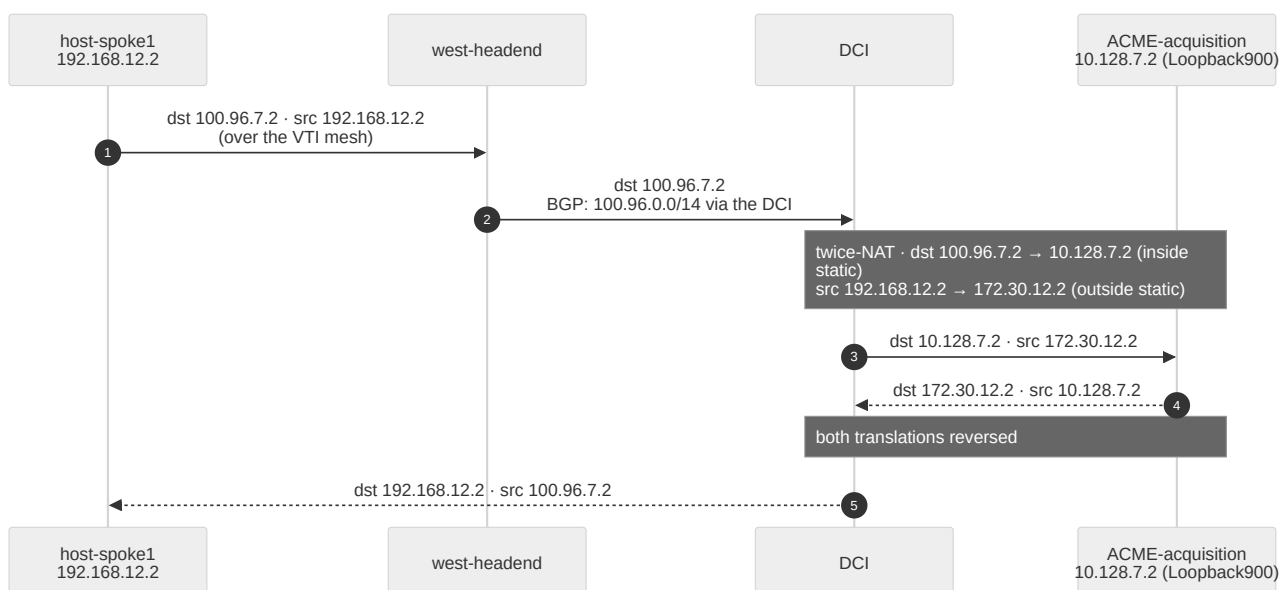
```
host-spoke1 $ traceroute -n 192.168.30.2          # the acquisition's LAN host
 1  192.168.12.1    0.407 ms    spoke1, the branch's own gateway
 2  172.17.4.1      1.508 ms    central-headend, over Tunnel4 (IPsec VTI)
 3  172.17.5.2      1.430 ms    spoke2 – the branch mesh is the transit between headends
 4  172.17.9.1      4.226 ms    west-headend, over Tunnel9
 5  *               the DCI does not answer this probe – see the note below
 6  100.64.21.2     4.056 ms    ACME-acquisition
 7  192.168.30.2    4.383 ms    host-acquisition
```

From a host that is *not* inside the overlapping prefix the same path is three hops shorter and the DCI answers normally:

```
host-west $ traceroute -n 192.168.30.2
 1  192.168.16.1    0.005 ms    west-headend
 2  100.64.20.2     0.523 ms    DCI
 3  100.64.21.2     1.201 ms    ACME-acquisition
 4  192.168.30.2    1.873 ms
```

Why hop 5 is silent — and why that is correct. The DCI's own routing table resolves `192.168.12.0/24` towards the acquired company (B `192.168.12.0/24` via `100.64.21.2`), because on the DCI that prefix belongs to the acquisition — ACME's copy is filtered out. A TTL-expired message the DCI generates for `host-spoke1` would therefore be sent to the acquisition's server instead of back to the branch, so it never arrives. Hosts inside the overlapping prefix cannot ping the DCI itself either (`host-west` can). Traffic through the DCI is unaffected: that is translated, and translation happens before the DCI's own routing decision matters.

The same path for a translated address, packet by packet



Neither end ever sees the other's real address, and neither end needs a route for it.

4. The routes exchanged (and the ones deliberately not)

One eBGP session per hop — the same `nautoBOT-bgp-models` peerings the tunnels use, just over a direct link:

```
DCI # show ip bgp summary
```

Neighbor	V	AS	MsgRcvd	MsgSent	TblVer	InQ	OutQ	Up/Down	State/PfxRcd
100.64.20.1	4	65206	1258	1266	219	0	0	17:15:07	14 west-headend (ACME)
100.64.21.2	4	65209	1146	1257	219	0	0	17:15:35	5 ACME-acquisition

What ACME learns from the DCI — seven prefixes, none of them colliding

```
west-headend # show ip bgp neighbors 100.64.20.2 routes
```

Network	Next Hop	Path	
*> 10.60.0.0/24	100.64.20.2	65208 65209 i	the acquisition's service prefix
*> 10.255.1.8/32	100.64.20.2	65208 i	DCI loopback
*> 10.255.1.9/32	100.64.20.2	65208 65209 i	acquisition loopback
*> 100.96.0.0/14	100.64.20.2	65208 i	the thousand overlapping prefixes, translated
*> 172.31.12.0/24	100.64.20.2	65208 i	the acquisition's server on the branch-1 LAN, translated
*> 192.168.30.0	100.64.20.2	65208 65209 i	the acquisition's LAN
*> 192.168.31.0	100.64.20.2	65208 i	the DCI's own LAN

Total number of prefixes 7

Those seven are re-advertised into the VTI mesh, so **every branch** has them through its own headends — with the AS path showing the trip out to the DCI:

```
spoke1 # show ip route bgp | include 100.96|172.31.12|192.168.30
```

```
B      100.96.0.0/14 [20/0] via 172.17.4.1, 03:41:50      via its tunnel to central-headend
B      172.31.12.0  [20/0] via 172.17.4.1, 03:41:50
B      192.168.30.0/24 [20/0] via 172.17.4.1, 03:41:50
```

```
spoke1 # show ip bgp 100.96.0.0/14
```

```
Paths: (2 available, best #2)
 65200 65203 65206 65208   from 172.17.1.1      east-headend → ... → west-headend → DCI
 65204 65202 65206 65208   from 172.17.4.1      central-headend → spoke2 → west-headend → DCI (best)
```

What the acquired company learns — twenty prefixes, ACME's branch-1 LAN excluded

```
ACME-acquisition # show ip route | include 10.128|100.96|100.100|192.168.12|192.168.30
```

```
S      10.128.0.0/14 is directly connected, Null0      the aggregate it originates for its own thousand
C      10.128.0.0/24 is directly connected, Loopback900 ...its own copies (+999 secondaries)
B      100.96.0.0/14 [20/0] via 100.64.21.1             how ACME sees the acquisition's thousand
B      100.100.0.0/14 [20/0] via 100.64.21.1            how the acquisition reaches ACME's thousand
C      192.168.12.0/24 is directly connected, Loopback2 its own copy – ACME's is never learned
C      192.168.30.0/24 is directly connected, GigabitEthernet5
```

The filter that keeps each copy at home

```
DCI # show running-config | include OVERLAP|NO-OVERLAP
ip prefix-list OVERLAP description prefixes that exist on both sides of the DCI: translated, not
advertised
ip prefix-list OVERLAP seq 5 permit 192.168.12.0/24
ip prefix-list OVERLAP seq 10 permit 10.128.0.0/14
route-map NO-OVERLAP deny 10 match ip address prefix-list OVERLAP
route-map NO-OVERLAP permit 20 everything else is advertised normally
neighbor 100.64.20.1 route-map NO-OVERLAP in ACME's copy never enters the DCI
neighbor 100.64.20.1 route-map NO-OVERLAP out the acquisition's copy never leaves towards ACME
neighbor 100.64.21.2 route-map NO-OVERLAP out ACME's copy never leaves towards the acquisition
```

Each router therefore keeps **exactly one path per prefix** — its own. What crosses the DCI instead is the translated range, which the DCI originates itself, together with the two static routes the translation needs:

```
DCI # show running-config | include ^ip route
ip route 100.96.0.0 255.252.0.0 Null0 name nat-inside-global translated before the
routing lookup
ip route 172.31.12.0 255.255.255.0 Null0 name nat-inside-global
ip route 100.100.0.0 255.252.0.0 100.64.20.1 name nat-outside-local routed before the translation
ip route 172.30.12.0 255.255.255.0 100.64.20.1 name nat-outside-local
```

Read the two halves. A packet from ACME to 100.96.x.y is translated *first* and then routed, so that range only needs to exist — a Null0 route is enough, and it is what the DCI advertises to ACME. A packet from the acquisition to 100.100.x.y is routed *first* and translated after, so the DCI needs a real next hop towards ACME. ACME itself never learns 100.100.0.0/14 : by the time a packet reaches west-headend its destination is already the real 10.128.x.1 .

5. How the NAT is set up

Two interface roles and two kinds of static translation — the pair that IOS calls *twice NAT*, because both the source and the destination of a flow are rewritten:

Term	In this lab	Configured as
inside local	the acquired company's real address, e.g. 10.128.7.2	ip nat inside source static network 10.128.N.0 100.96.N.0 /24
inside global	the same host as ACME sees it: 100.96.7.2	
outside global	ACME's real address, e.g. 10.128.7.1 (or 192.168.12.2)	ip nat outside source static network 10.128.N.0 100.100.N.0 /24
outside local	the same host as <i>the acquisition</i> sees it: 100.100.7.1 (or 172.30.12.2)	

```
DCI # show running-config | include source static network 10.128.0.0 |source static network 192.168.12.0
ip nat inside  source static network 10.128.0.0   100.96.0.0 /24      × 1000, one per scale prefix
ip nat outside source static network 10.128.0.0   100.100.0.0 /24      × 1000
ip nat inside  source static network 192.168.12.0 172.31.12.0 /24      the single hand-written overlap
ip nat outside source static network 192.168.12.0 172.30.12.0 /24

DCI # show ip nat statistics
Total active translations: 3018 (2002 static, 1016 dynamic; 0 extended)
Outside interfaces:  GigabitEthernet2          towards ACME
Inside interfaces:   GigabitEthernet3          towards the acquired company
Hits: 21266 Misses: 19603
In-to-out drops: 0 Out-to-in drops: 0
```

A `network static` maps a whole /24 in one line and shows up in the translation table as a single entry per address in use:

```
DCI # show ip nat translations | include 100.96.7.2
--- 100.96.7.2      10.128.7.2      ---
    inside global   inside local   outside local   outside global
```

Where the thousand come from

One `scale` block in `lab-intent.json` generates every line above — the statics, the loopback addresses on both sides, the aggregates and the DNS records:

```
"nat": { "inside_interface": "GigabitEthernet3", "outside_interface": "GigabitEthernet2",
"dns_fixup": true,
  "overlaps": [ { "prefix": "192.168.12.0/24", "inside_global": "172.31.12.0/24", "outside_local":
"172.30.12.0/24" } ],
  "scale": { "count": 1000, "prefix_base": "10.128.0.0/24",
    "inside_global_base": "100.96.0.0/24", "outside_local_base": "100.100.0.0/24",
"aggregate_len": 14,
    "acme_router": "west-headend",      "acme_host": 1,
    "acquisition_router": "ACME-acquisition", "acquisition_host": 2,
    "loopback_base": 900, "loopback_mode": "secondaries" } }
```

Generated	On the device	Note
2 × 1000 static translations	DCI	plus 2 for the single overlap = 2002
1000 addresses per side	west-headend and ACME-acquisition, Loopback900	primary + 999 secondary addresses — one interface, not a thousand
3 aggregates	BGP	100.96.0.0/14, 100.100.0.0/14, 10.128.0.0/14 — the routing table stays small
1 prefix-list line	DCI	OVERLAP seq 10 permit 10.128.0.0/14 covers all thousand
2 × 1000 DNS records	the two zones (§6)	one per prefix, pointing at that side's own copy

Two platform limits worth knowing. IOS-XE's CLI RPC rejects a payload of this size (*“invalid value for: config-clis ... has a bad length/size”* at roughly two thousand lines), so the renderer splits every CLI template into **400-line chunks**, repeating the section header where the lines only make sense inside one. And a CLI template never removes the lines of a previous version — shrinking a scale set means removing the old statements explicitly.

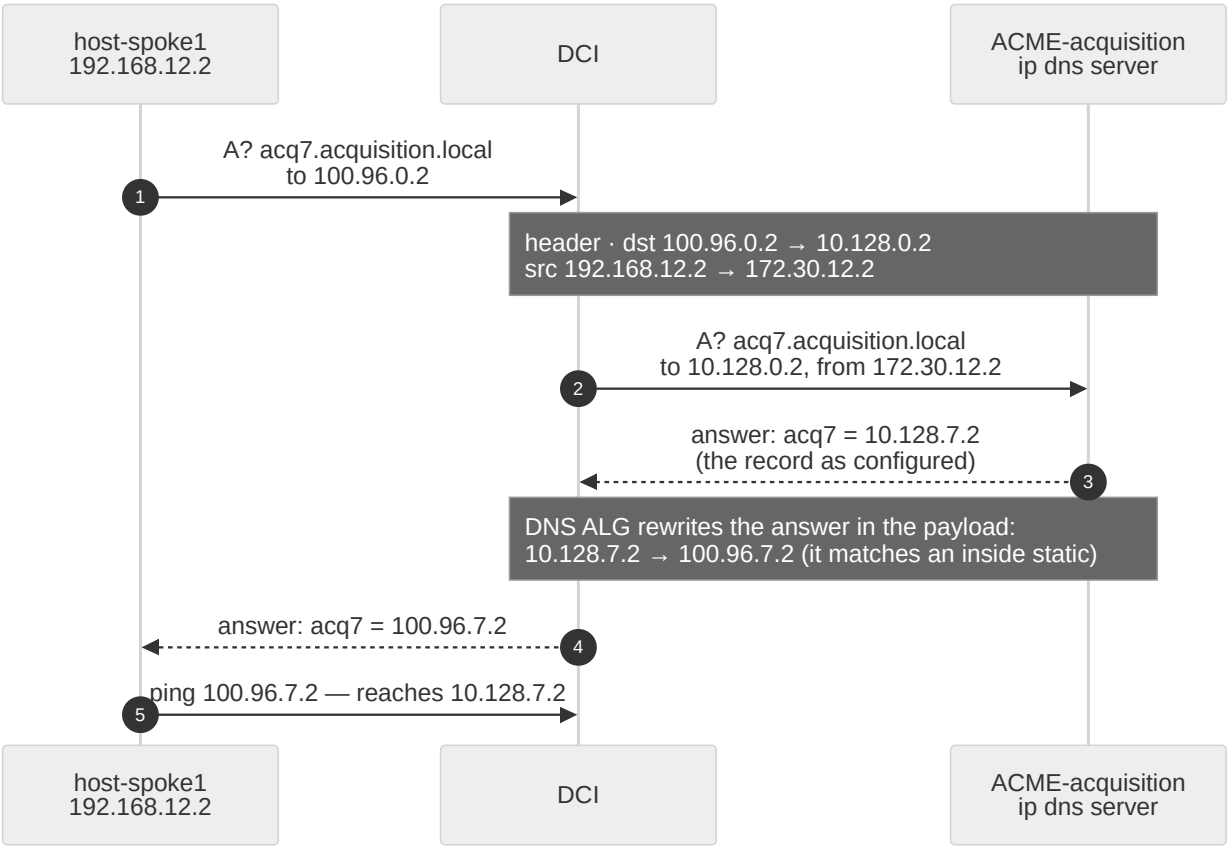
6. How DNS is fixed up

Each side answers for its own zone, with its own copies of the overlapping addresses. A resolver on the far side of the DCI would therefore be handed an address that means something completely different at home — unless the NAT rewrites the answer. That rewrite is the DNS **application-level gateway** built into IOS NAT: when a DNS reply crosses a NAT boundary, the A records inside the payload are translated with the same statics as the packet header.

Zone	Answered by	Records	Asked by
acquisition.local	ACME-acquisition ip dns server + 1001 ip host lines	acq0...acq999 → 10.128.N.2 server → 10.128.0.2	host-spoke1, whose resolv.conf names the server at its <i>translated</i> address
acme.local	west-headend	svc0...svc999 → 10.128.N.1 plus the named lab hosts	ACME-acquisition itself (ip name-server + ip domain lookup source-interface)

```
host-spoke1 $ cat /etc/resolv.conf
search acquisition.local
nameserver 100.96.0.2
```

written by cloud-init from `dns\_client` in the intent
= 10.128.0.2 (the server) as ACME sees it



```

host-spoke1 $ nslookup -type=a acq777.acquisition.local
Address: 100.99.9.2          the record on the server says 10.131.9.2

DCI # show ip nat translations | include :53
      inside global      inside local      outside local      outside global
branch → the acquisition's zone
udp    100.96.0.2:53      10.128.0.2:53      172.30.12.2:51258  192.168.12.2:51258
the acquisition → ACME's zone
udp    172.31.12.2:38993  192.168.12.2:38993  192.168.16.1:53    192.168.16.1:53

```

The reverse direction works the same way and is easiest to see from the router itself, which resolves and pings in one command:

```

ACME-acquisition # ping svc7.acme.local source 10.128.7.2
Sending 2, 100-byte ICMP Echos to 100.100.7.1, timeout is 2 seconds:  the record says 10.128.7.1
Packet sent with a source address of 10.128.7.2
!!
Success rate is 100 percent (2/2), round-trip min/avg/max = 1/2/3 ms

```

The source address is the part that is easy to get wrong. The ALG only rewrites a payload on a flow the NAT is already translating. `host-spoke1` sits in `192.168.12.0/24` — the single hand-written overlap — so its queries leave as `172.30.12.2` and the answer comes back doctored; `ACME-acquisition` sources its own queries from `Loopback2` for exactly the same reason. A host outside any overlap would receive the raw `10.131.9.2` and have nowhere to send it. Setting `"dns_fixup": false` in the intent renders the opposite: `no-payload` on the statics and `no ip nat service dns tcp/udp`.

7. How the test is run and validated

Suite `tests/suites/12_dci.robot` — twelve tests, every expected value read from the intent or Nautobot, never from a constant in the test. Run it with `./lab.sh test` (or from the portal); the whole suite is 74 tests across 12 suites.

Test	What it proves	Time
The DCI chain is modelled in Nautobot	roles <code>vpn-dci / partner-edge</code> , the right tenants and design patterns, both links cabled with their /30s, the acquisition's service prefix tagged for advertisement	1s
The interconnect links are up ... no IPsec anywhere	modelled addresses on both ends, the far end pings, and neither router holds a tunnel, crypto profile, keyring or trustpoint	8s
eBGP is Established on every hop	one session per link, over the link addresses, with the AS numbers from the model	4s
The acquisition originates its prefixes and every branch learns them	<code>network</code> statements on the acquisition; every spoke has the prefixes via one of its own headends	7s
A branch reaches the acquisition's LAN and its service prefix	end-to-end ping from each branch's LAN address, and the acquisition back to every branch	8s
The host behind the acquisition is in the mesh and breaks out	full host ping mesh plus internet through <code>west-headend</code> , with the DCI and headend hops asserted in the traceroute	9s
Golden Config renders the DCI chain and reports it compliant	both routers in the Golden Config scope; every compliance feature matches, including the 2002 NAT lines and the 1000 DNS records	0s
The overlapping prefix exists on both sides and neither learns the other's copy	both sides really do use <code>192.168.12.0/24</code> ; the DCI reaches it through the acquisition; no spoke learns the acquisition's copy and vice versa	6s
The DCI translates in both directions	ACME reaches the server at the inside-global address, the acquisition reaches ACME's host at the outside-local one, and both halves appear in the translation table	8s
DNS is fixed up on the way through the NAT	a name whose record is an overlapping address resolves to the translated one, and the address answers	3s
NAT scales : a thousand prefixes, both directions, no drops	2002 static translations, zero drops, only the three aggregates in the routing table, and a branch host reaching 1000/1000 translated prefixes	10s
DNS fix-up scales and works from either side	resolved+reached 1000/1000 from the branch host, a sample of the reverse direction from the acquisition, and a translated DNS flow in the NAT table	16s

How the two scale tests actually sweep

1. The expected values are computed from the intent, not hard-coded: `intent.nat_scale()` expands the `scale` block into 1000 entries, each with its prefix, its inside-global and outside-local forms, and

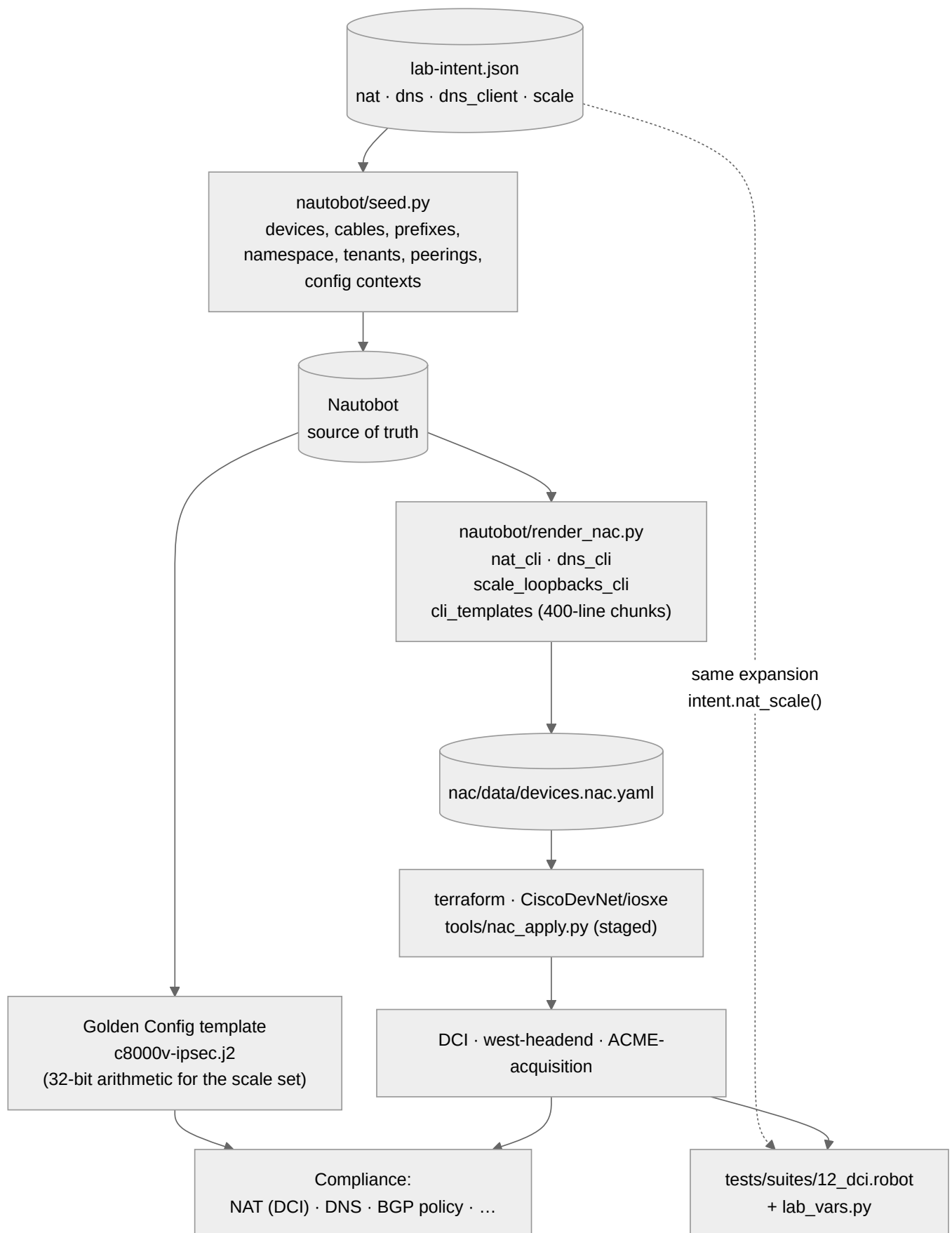
each side's address (`acquisition_as_acme_sees_it = 100.96.N.2`, `acme_as_acquisition_sees_it = 100.100.N.1`). The same function feeds the renderer, so the test and the device are computed from one source.

2. **The NAT sweep** ships the thousand target addresses to `host-spoke1` as one list and pings them forty at a time (`xargs -P 40`): about five seconds for all thousand, against minutes if run serially. The test asserts `reached 1000/1000` and prints any address that failed.
3. **The DNS sweep** ships a thousand `name=expected` pairs, resolves each with `nslookup -type=a` (filtering out busybox's server line and its AAAA “No answer” noise), compares the answer to the address the NAT *should* have produced and then **pings it** — so a name that resolves to something unreachable fails. The assertion is `resolved+reached 1000/1000`.
4. **The reverse direction** is checked from the router, where there is no busybox: `ping svcN.acme.local source 10.128.N.2` must both resolve to the outside-local form and succeed.
5. **The counters** are read independently: `show ip nat statistics` must show exactly two static translations per overlapping prefix and zero drops, and `show ip route` must contain the three aggregates and nothing per-prefix.

What a run leaves behind

Artefact	Contents
<code>results/<run>/report.pdf</code>	the evidence report: every suite and test with its result, duration and the numbers it measured (<code>reached 1000/1000</code> , <code>resolved+reached 1000/1000</code> , <code>Total active translations: ... 2002 static</code>)
<code>results/<run>/log.html</code> , <code>report.html</code> , <code>output.xml</code>	Robot's own log and report; the PDF is generated from the XML, so it can be rebuilt for any past run
<code>results/<run>/configs/</code>	running and startup configuration of every Catalyst 8000v before and after the run, and <code>pre-vs-post.diff</code>
Nautobot Golden Config	171/171 compliance rows compliant — the NAT and DNS are two of the compliance features, so a hand-edit anywhere in the 2002 lines or the 2000 records shows up as drift

8. Where every line of this comes from



File	Its part in the DCI
------	---------------------

<code>lab-intent.json</code>	the <code>nat</code> , <code>dns</code> and <code>dns_client</code> blocks, including the <code>scale</code> block that generates the thousand
<code>nautobot/intent.py</code>	<code>nat_scale()</code> , <code>nat_scale_aggregates()</code> , <code>dns_scale()</code> — the one expansion used by the renderer, the template context and the tests
<code>nautobot/seed.py</code>	the devices, roles, tenants, cables, prefixes (in the <code>acquisition</code> namespace), BGP peerings and the config contexts keyed by hostname
<code>nautobot/render_nac.py</code>	the NAT and DNS CLI templates, the loopback-secondaries block, the interface <code>ip nat inside/outside</code> flags, the prefix-list and route-map, and the 400-line chunking
<code>nautobot/golden-config-templates/c8000v-ipsec.j2</code>	the intended configuration for the same lines — expanded in 32-bit arithmetic so the thousand prefixes stay correct past the third octet
<code>tests/suites/12_dci.robot</code> , <code>tests/resources/lab_vars.py</code>	the twelve tests and the values they expect, derived from the intent
<code>tests/report_pdf.py</code> , <code>tests/run.sh</code>	the evidence PDF each run produces